COMP2611 Spring 2026 Homework #3

(Deadline 11:55pm, Thursday April 16, 2026 HKT, UTC+8)

Notes:

- This is an individual assignment; all works must be your own. You can discuss with your friends but never show your code to others. Please also **do not post any code to piazza**.
- Write your code in given MIPS assembly skeleton files. Add your own code under TODOs in the skeleton code. Keep other parts of the skeleton code unchanged.
- Make procedure calls with the registers as specified in the skeleton, otherwise the provided procedures may not work properly. Preserve registers according to the MIPS register convention on slide 76 of the ISA note set.
- Zip the two finished MIPS assembly files into a single zip file, *<your_stu_id>.zip* file (without the brackets). Do not change names of the given skeleton files.
- To submit, first find the Canvas page of COMP2611, homework 3, and then upload the "*<your_stu_id>.zip*" file. You can upload for as many times as you like, only the latest one before the deadline will be marked.
- **Your submitted program must be able to run under MARS, otherwise it will not be marked**.

No late submission will be accepted.

Question 1: Delta Encoding and Decoding (20 marks)

In data compression and signal processing, **delta encoding** stores data as differences (deltas) between consecutive values rather than storing the full data directly. Given the original array $A[1 \ldots n]$, its delta encoded form $\Delta[1 \ldots n]$ is defined as:

- $\Delta[1] = A[1]$
- For $i > 1, \Delta[i] = A[i] - A[i-1]$

To restore the original array from Δ, we perform delta decoding:

- $A[1] = \Delta[1]$
- For $i > 1, A[i] = A[i-1] + \Delta[i]$

For example, if $n = 6$ and $A = [10, 13, 13, 8, 9, 20]$, then $\Delta = [10, 3, 0, -5, 1, 11]$. Decoding this Δ will restore the original array A.

Your task is to write a MIPS assembly program to implement **delta decoding** (i.e., compute A from the given Δ). The procedure should implement the same functionality as the `DeltaDecoding()` function in the C++ program given on the next page.

Add your own code under the "TODO" in the skeleton code. Follow the instructions specified in the "TODO". DO NOT modify other parts of the skeleton code. NO new procedure is allowed. But whenever needed, you can add label(s) under TODO. Properly comment your MIPS code. For simplicity you can assume the **maximum length of A[] to be 100**, and all the input elements of array `A[]` are valid signed integers. You can also assume that there is no overflow in all the cumulative sums.

C++ Program

```cpp
#include <iostream>
using namespace std;

// Decode delta-encoded array in-place:
// after the function returns, delta[] becomes the decoded array A[].
void DeltaDecoding(int delta[], int A_SIZE) {
    // A[0] = delta[0], so the first element stays unchanged.
    // For i >= 1: A[i] = A[i-1] + delta[i]
    for (int i = 1; i < A_SIZE; i++) {
        delta[i] = delta[i - 1] + delta[i];    // overwrite delta[i] with decoded A[i]
    }
}

int main() {
    const int A_SIZE = 10;
    int delta[100];     // delta[] can hold up to 100 elements

    // Read delta[] from standard input
    cout << "Please enter integers in delta array delta[] one by one, use [Enter] to split:" << endl;
    for (int i = 0; i < A_SIZE; i++) {
        cout << "delta[" << i << "]: ";
        cin >> delta[i];
    }

    // Decode in-place (delta[] becomes A[])
    DeltaDecoding(delta, A_SIZE);

    // Output the decoded array
    cout << "The decoded array A[] is: ";
    for (int i = 0; i < A_SIZE; i++) {
        cout << delta[i] << " ";    // delta[i] now stores A[i]
    }
    cout << endl;

    return 0;
}
```

A sample execution of the program in MARS

```
Please enter integers in delta array delta[] one by one, use [Enter] to split:
delta[0]: 9
delta[1]: -2
delta[2]: -3
delta[3]: 4
delta[4]: 2
delta[5]: 1
delta[6]: -6
delta[7]: 2
delta[8]: 1
delta[9]: -3
The decoded array A[] is: 9 7 4 8 10 11 5 7 8 5

-- program is finished running --
```

Question 2: Integral Image (30 marks)

In image processing and data analysis, we often need to compute the sum of values inside many rectangular regions of a 2D grid (e.g., the total brightness in a region of an image).

An **Integral Image** (also called a **2D prefix sum table**) is a precomputed matrix that allows fast region-sum computation. For an input matrix A, the integral image S stores cumulative sums from the top-left corner so that each entry represents the sum of a rectangular area.

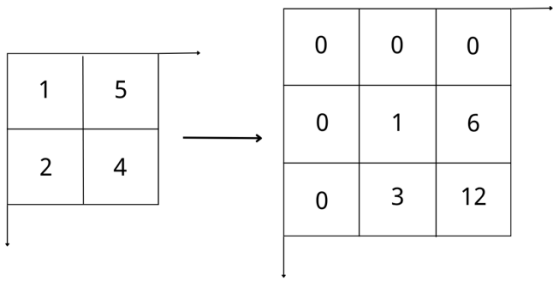
In this question, you will construct an integral image S from a given input matrix A. Assume A is a $N \times M$ matrix indexed from $(1,1)$ to (N,M). The integral image S is an $(N+1) \times (M+1)$ matrix defined as:

$S[0][j] = 0$ for all $j = 0..M$ (i.e. the top row of S[][] is holding 0's)
$S[i][0]$ = 0 for all $i = 0..N$ (i.e. the left column of S[][] is holding 0's)
$For\ i = 1..N$ and $j = 1..M$:

$$S[i][j] = A[i][j] + S[i-1][j] + S[i][j-1] - S[i-1][j-1]$$

Note that to compute ***S[i][j]*** we need to have ***S[i-1][j]***, ***S[i][j-1]*** and ***S[i-1][j-1]*** computed first. All computations are done using signed 32-bit integers.

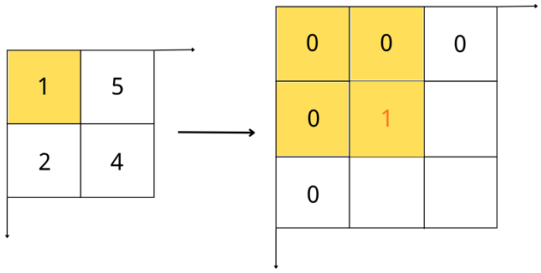
For example, given the following 2D array:



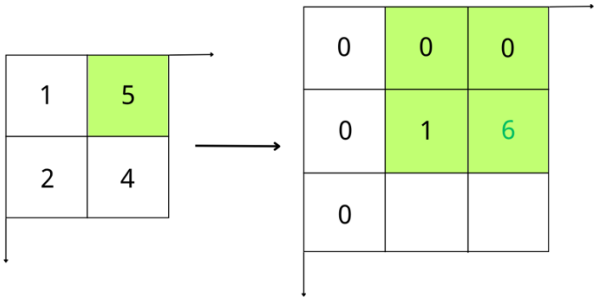
input image integral image

As an illustration, the detailed calculation steps for each of the $S[][]$ elements are shown below:

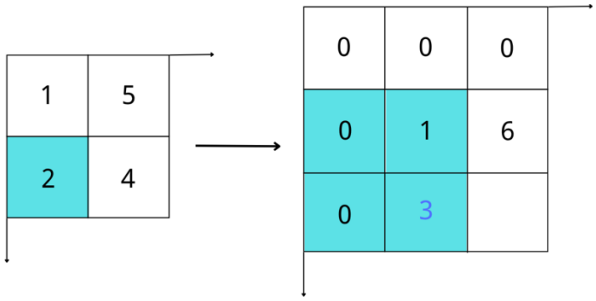
$$S[1][1] = A[1][1] + S[0][1] + S[1][0] - S[0][0] = 1 + 0 + 0 - 0 = 1$$



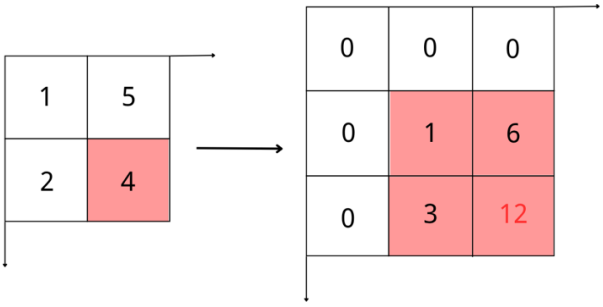
$$S[1][2] = A[1][2] + S[0][2] + S[1][1] - S[0][1] = 5 + 0 + 1 - 0 = 6$$



$$S[2][1] = A[2][1] + S[1][1] + S[2][0] - S[1][0] = 2 + 1 + 0 - 0 = 3$$



$$S[2][2] = A[2][2] + S[1][2] + S[2][1] - S[1][1] = 4 + 6 + 3 - 1 = 12$$



Your task is to implement the `integral` MIPS procedure in skeleton file. This procedure should implement the same functionality as the C++ procedure `integral()` in the C++ program.

Add your own code under the "TODO" in the skeleton code. Follow the instructions specified in the "TODO". DO NOT modify other parts of the skeleton code. NO new procedure is allowed. But whenever needed, you can add label(s) under TODO. Properly comment your MIPS code. You can assume the matrix A[] is stored in MIPS as a 1-D array `A[]` in row order (under the label "`A`"). For example, the following matrix:

$$\begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,m} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,m} \end{pmatrix}$$

is stored in a 1-D array in the row order in MIPS as:

$$A[\,] = \{a_{1,1}, a_{1,2}, \ldots, a_{1,m}, a_{2,1}, a_{2,2}, \ldots, a_{2,m}, \ldots, a_{n,1}, a_{n,2}\ , \ldots, a_{n,m}\}$$

Note that the index of A[] starts from 1 instead of 0, so the first element of A[] is really A[1][1] = $a_{1,1}$ instead of A[0][0].

And matrix S as

$$S = \begin{pmatrix} 0 & 0 & \cdots & 0 \\ 0 & s_{1,1} & \cdots & s_{1,m} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & s_{n,1} & \cdots & s_{n,m} \end{pmatrix}$$

You can assume that the maximum number of elements in A[] is 25 (i.e. `n*m <= 25`), and the maximum number of elements in S[] is 36 (i.e. `(5+1)*(5+1) = 36`)

C++ Program

```
#include <iostream>
using namespace std;

int multiply(int a, int b)
{
	int result = 0;
	for (int i = 0; i < b; i++)
	{
		result = result + a;
	}
	return result;
}

void integral(const int A[], int S[], int N, int M)
{
	//each row of the S[][] array has M+1 elements (i.e. 1 extra element more than A[][]),
	//we need this to calculate for the S[i][j] address
	int SM = M + 1;

	// row 0, init row 0 of S[][] with 0's
	for (int j = 0; j <= M; j++) S[j] = 0;
	// col 0, init col 0 of S[][] with 0's
	for (int i = 0; i <= N; i++) S[multiply(i, SM)] = 0;

	for (int i = 1; i <= N; i++)
	{
		for (int j = 1; j <= M; j++)
		{
			// row major address calculation for A[i][j], note that i and j of A[][] both start from 1, NOT 0
			int aIdx     = multiply(i - 1, M)    + (j - 1);

			// row major calculation for the address of S[i][j]
			int sIdx     = multiply(i, SM)         + j;

		// row major calculation for the address of S[i-1][j]
		int upIdx    = multiply(i - 1, SM) + j;
		// row major calculation for the address of S[i][j-1]
		int leftIdx= multiply(i, SM)         + (j - 1);
		// row major calculation for the address of S[i-1][j-1]
		int diagIdx= multiply(i - 1, SM) + (j - 1);

		// calculate S[i][j]= A[i][j]+S[i-1][j]+S[i][j-1]-S[i-1][j-1] according to the equation
		S[sIdx] = A[aIdx] + S[upIdx] + S[leftIdx] - S[diagIdx];
```

```
            }
        }
}

void print2DMatrix(const int X[], int R, int C)
{
        for (int i = 0; i < R; i++)
        {
            for (int j = 0; j < C; j++)
            {
                if (j > 0) cout << " ";
                cout << X[multiply(i, C) + j];
            }
            cout << endl;
        }
}

int main()
{
        int N, M;

        cout << "Enter the number of rows N: ";
        cin >> N;
        cout << "Enter the number of columns M: ";
        cin >> M;

        int A[25];
        int S[36];

        cout << "Enter the input image A in row-major order:" << endl;
        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < M; j++)
            {
                cout << "A[" << i+1 << "][" << j+1 << "]: ";
                cin >> A[multiply(i, M) + j];
            }
        }

        cout << "The initial matrix is " << endl;
        print2DMatrix(A, N, M);

        integral(A, S, N, M);
```

```
    cout << "Here is the integral image S:" << endl;
    print2DMatrix(S, N + 1, M + 1);

    return 0;
}
```

Sample execution of the program in MARS

```
Enter the number of rows N: 2
Enter the number of columns M: 2
Enter the input image A in row-major order:
A[1][1]: 1
A[1][2]: 5
A[2][1]: 2
A[2][2]: 4
The initial matrix is
1 5
2 4
Here is the integral image S:
0 0 0
0 1 6
0 3 12
```